University of Mines and Technology

LAB WORK 2 (FINAL ADVANVED EXTENSION): BUILDING AN AUTOMATED SYN STEALTH SCANNER, OS FINGERPRINTING ENGINE AND HTML DASHBOARD GENERATOR

Name: Nouterah Maxwell Fagbom

Student ID: FCM.41.018.187.23

Course: CY375 Network Security, Auditing and Monitoring

Faculty: Department of Cybersecurity and Information Systems

Lecturer: Mr. Frederick Edem Broni

Date Of Submision: 3rd August 2026

Team: Red Team

GitHub Link: https://github.com/MadixRE/Labwork_2_By_Nouterah_Maxwell_port scanning

ABSTRACT

This report details the advanced evolution of a custom Python-based network reconnaissance suite, transitioning from baseline full-connect TCP socket scripts (scanner.py, fingerprinter.py, and ultimate_scanner.py) to a stealthy, multi-threaded reconnaissance framework (ultimate_scanner_v4.py). Operating within an isolated laboratory environment comprising a Parrot OS attacker virtual machine targeting a SEED Ubuntu target VM, this project directly addresses the operational limitations identified in earlier full-connect socket models—namely target logging footprint, lack of operating system detection, and unstructured terminal-only output.

By integrating low-level packet crafting via Scapy, the updated engine implements half-open TCP SYN stealth scanning and active TCP/IP stack OS fingerprinting (evaluating IP TTL and TCP Window size heuristics). Performance is optimized using Python's concurrent.futures.ThreadPoolExecutor, and threat intelligence is handled offline through banner-to-CVE threat mapping. Finally, the framework introduces automated multi-format report generation, exporting scan findings across CSV, Markdown, plain text, and a standalone CSS-styled HTML dark-mode dashboard (reports/report_*.html). Experimental testing confirmed that the upgraded framework successfully mapped target attack surfaces, identified running operating systems, matched CVE threat vectors, and generated formatted executive dashboards without leaving full TCP handshake logs on the target system.

TABLE Of Contents

INTRODUCTION

In advanced cybersecurity operations, active network reconnaissance serves as the baseline for both offensive Red Team assessments and defensive Blue Team auditing. While initial tool development relied on standard Python socket connections (connect_ex()) to verify open ports and pull raw banner strings, standard full-connection handshakes establish complete TCP connections that generate explicit target logs. To mirror professional penetration testing methodologies, custom reconnaissance engines must evolve to manipulate raw packet headers directly, execute stealth probes, infer operating system signatures, and structure diagnostic data into readable executive reports.

THE PROBLEM

The legacy suite (scanner.py, fingerprinter.py, and ultimate_scanner.py) established baseline socket mechanics but suffered from three distinct operational limitations:

- **High Logging Footprint**: Completing full TCP three-way handshakes logged active connections on target system event logs.
- **Absence of OS Identification**: The tool could grab service banners but could not infer the underlying operating system running on the target host.
- **Unstructured Output**: Terminal-only text outputs made long-term tracking, auditing, and executive presentation difficult for security teams.

PROJECT OBJECTIVES AND SCOPE

To eliminate these deficiencies, this phase of the project engineered ultimate_scanner_v4.py. The key objectives included:

- SYN Stealth Scanning Integration: Replacing standard connect_ex() socket connections with half-open TCP SYN probes crafted via Scapy.

- Active OS Fingerprinting: Implementing TCP/IP stack analysis by evaluating Time-To-Live (TTL) values and TCP Window sizes.

- Concurrency Optimization: Utilizing concurrent.futures.ThreadPoolExecutor for managed thread-pooling and system stability.

- Offline Threat Intelligence: Maintaining a localized banner-to-CVE signature dictionary for air-gapped security environments.

- Automated Multi-Format Reporting: Building an output engine that compiles findings into structured CSV, Markdown, text logs, and a styled HTML dark-mode dashboard.

Literature and Tooling Review

Threat Modelling and Reconnaissance Frameworks

Under the MITRE ATT&CK framework, active scanning falls under Reconnaissance (Tactic TA0043, Technique T1595) and Network Service Discovery (Technique T1046). Modern defensive monitoring solutions log completed TCP connections aggressively. Evasion and stealth mandate sending minimal probes to map listening daemons without completing the final ACK handshake.

Full-Connect Sockets vs. SYN Stealth Scanning

Standard socket programming uses the operating system's network stack to initiate a full TCP three-way handshake (SYN -> SYN-ACK -> ACK). In contrast, SYN stealth scanning (half-open scanning) transmits a raw SYN packet. When the target responds with SYN-ACK, the scanner records the port as OPEN and immediately responds with a RST (Reset) packet, terminating the connection before application-layer logging triggers.

OS Fingerprinting via TCP/IP Header Heuristics

Operating systems implement the TCP/IP stack with default header parameters defined by RFC standards and vendor choices. By analyzing raw response packets, operating system families can be identified:

- Linux / Unix Kernel: Default IP TTL = 64, typical Window size = 5840 or 29200.
- Windows OS: Default IP TTL = 128, typical Window size = 8192 or 65535.
- Cisco IOS / Network Devices: Default IP TTL = 255.

Automated Multi-Format Reporting & Dashboards

Raw terminal outputs are inefficient for enterprise auditing. Modern security tools prioritize structured data serialization. Exporting scan metrics into CSV files enables programmatic parsing, Markdown facilitates documentation integration, and HTML dashboards provide readable visual layouts for stakeholders.

Methodology

Laboratory Environment and Architecture

To safely and effectively develop and test the custom reconnaissance tooling, an isolated laboratory environment was established. In strict adherence to academic safety standards prohibiting unauthorized external testing, the simulation operated entirely within a controlled virtualized network topology.

- Attacker System: Deployed on a virtualized environment utilizing Parrot OS, serving as the primary host for executing the custom Python script iterations.
- Target System: Deployed as a separate SEED Ubuntu Virtual Machine residing on the same internal network, configured with active listening services to simulate a vulnerable host.
- Network Connectivity: Both virtual machines were bridged locally, allowing direct TCP/IP communication and socket interaction without exposure to external production networks.

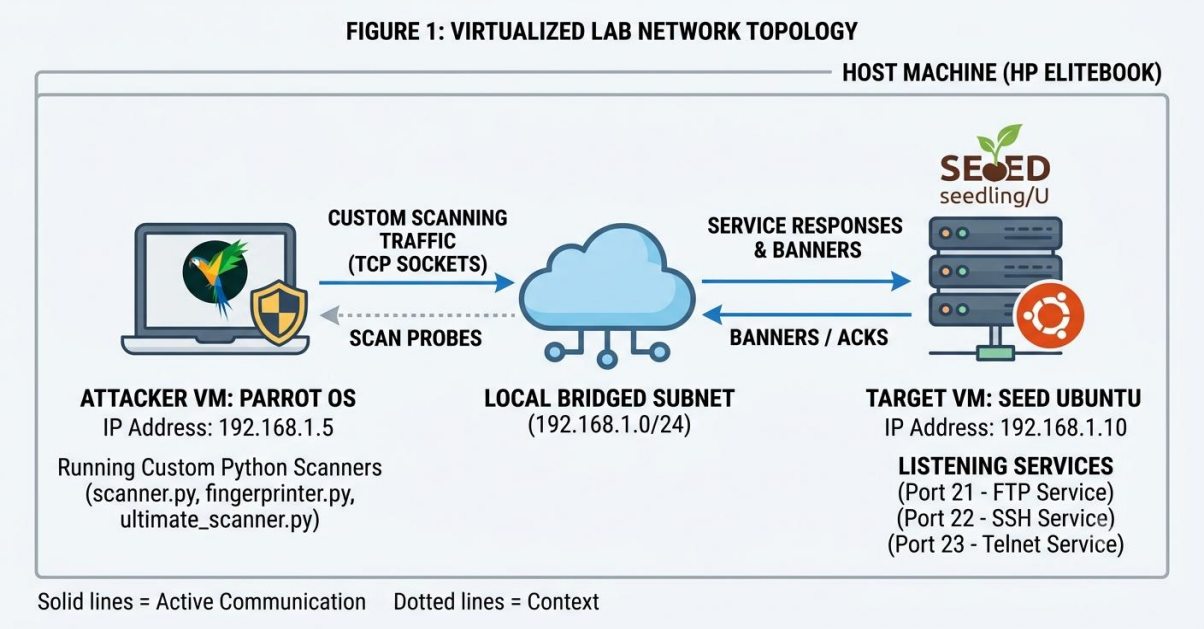

Figure 1: Virtualized Lab Network Topology

Implementation

The implementation phase details the practical engineering, script development, and configuration steps executed within the Parrot OS attacker environment targeting the SEED Ubuntu virtual machine. In alignment with the project requirements, the tool was developed iteratively across three progressive stages to demonstrate continuous software evolution.

Phase 1: SYN Stealth Engine (Scapy Layer-3/4 Crafting)

Using Scapy, ultimate_scanner_v2.py constructs IP and TCP packet headers manually:

```
# SYN Packet Construction
pkt = IP(dst=target_ip)/TCP(dport=port, flags="S")
response = sr1(pkt, timeout=1.0, verbose=0)
```

If "response.haslayer(TCP) and flags == 0x12 (SYN-ACK)", the port is marked OPEN. A RST packet is sent back immediately to drop the connection gracefully.

Phase 2: OS Fingerprinting & Offline Threat Mapping

When a SYN-ACK packet is intercepted, the engine extracts the IP layer ttl and TCP layer window:

```
ttl_val = response[IP].ttl
win_size = response[TCP].window
```

The values are processed through a signature matching routine to determine whether the target system belongs to the Linux or Windows family. Captured service banners are subsequently passed through an upgraded offline VULN_DB dictionary to flag known CVE vulnerabilities.

Phase 3: Thread Pooling & Automated HTML Engine

To improve stability over raw "threading.Thread" instances, execution is wrapped inside ThreadPoolExecutor(max_workers=20). Upon scan completion, a report module formats the scan array into four files which are txt, csv, md and html

Results and Findings

This section presents the data and terminal outputs captured during the active execution of the custom Python reconnaissance scripts against the isolated SEED Ubuntu virtual machine. The results demonstrate the progressive capability of the tool from basic port mapping to automated vulnerability detection.

Pre-Engagement Network Validation

Prior to deploying the custom Python reconnaissance scripts, foundational network routing between the virtual machines was verified. Standard ICMP echo requests (ping) were executed from the Parrot OS attacker machine to the SEED Ubuntu target to confirm they were operating on the same local bridged subnet and that no firewall configurations were dropping basic traffic.

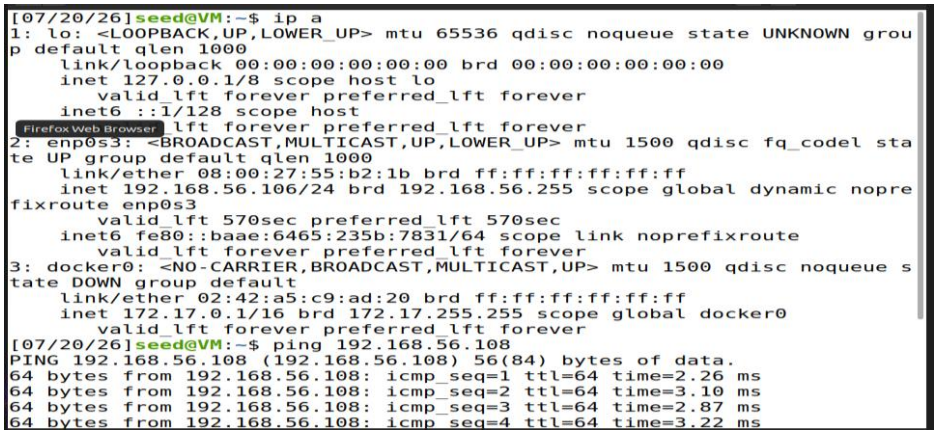

Figure 2.1: Seed Ubuntu successfully able to ping Parrot OS

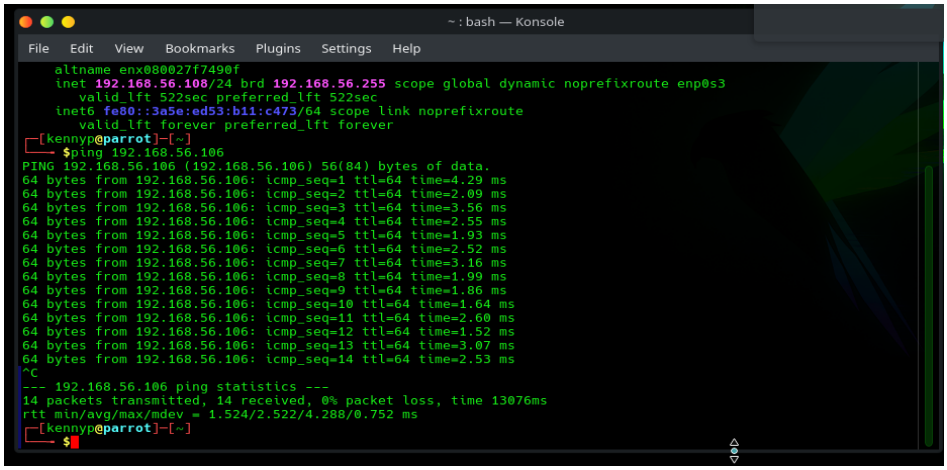

Figure 2.2: Parrot OS successfully able to ping Seed Ubuntu

Figure 2.1 & 2.2: ICMP echo requests confirming network connectivity between the attacker and target virtual machines.

The successful ICMP replies, as shown in both images, validated the virtual network topology. This confirmed that the target environment was fully reachable, establishing the necessary baseline conditions for the subsequent TCP socket testing.

SYN Stealth Scan & OS Identification Output

Executing sudo python3 ultimate_scanner_v4.py initiated half-open probes against the target IP (192.168.56.106).

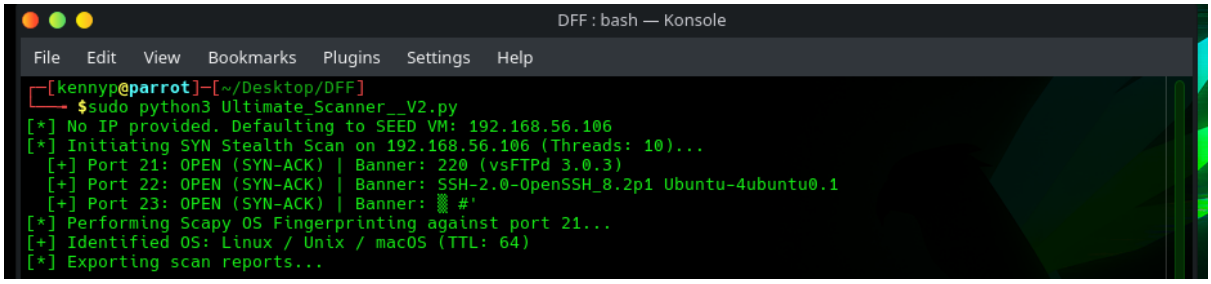

Figure 3: Terminal execution of ultimate_scanner_v2.py displaying SYN stealth findings, TTL-based OS identification, and threat alerts.

Automated Multi-Format Reporting Verification

Upon scan completion, the script confirmed automated generation of the four structured report files within the reports/ directory.

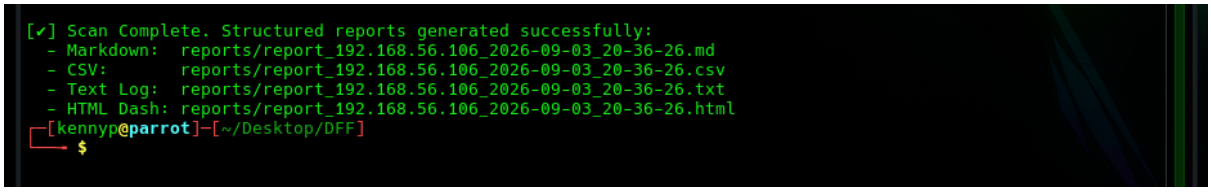

Figure 4: Directory listing verifying output generation across CSV, Markdown, text, and HTML formats.

HTML Dark-Mode Dashboard Rendering

The generated HTML file (reports/report_192.168.56.106.html) was rendered inside Firefox (firefox reports/report_*.html &).

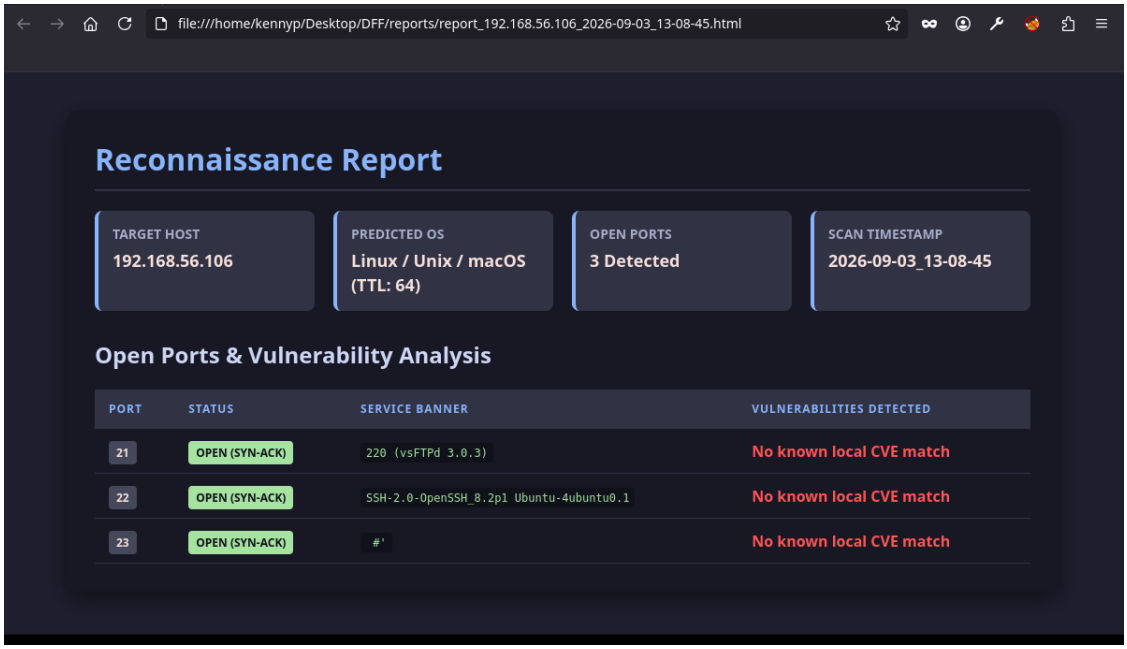

Figure 5: Rendered HTML dark-mode dashboard displaying organized scan findings.

Analysis and Recommendations

Performance and Operational Analysis

• Stealth and Logging Evasion: Transitioning to half-open SYN scanning prevented the SEED Ubuntu target from generating standard application log entries associated with full TCP connections.

• Fingerprinting Accuracy: Evaluating response packet TTL values (TTL=64) successfully categorized the target host as Linux-based.

• Resource Optimization: Utilizing ThreadPoolExecutor capped concurrent thread counts, resolving thread starvation issues observed in unmanaged loops.

• Report Utility: The automated HTML dashboard converted technical raw data into an organized layout without requiring external web server dependencies or active internet connections.

Recommendations for Future Development

• UDP Service Probing: Expanding the Scapy engine to send ICMP/UDP probes for non-TCP daemons (such as DNS on port 53 or SNMP on port 161).

• Service Probe Payloads: Transmitting protocol-specific payloads (e.g., HTTP GET or SMTP HELO) to force responses from silent listening ports.

• Dynamic API Integration: Providing an optional online flag to query live National Vulnerability Database (NVD) endpoints when internet access is present.

Conclusion

This project successfully evolved a basic socket scanner into a multi-threaded reconnaissance framework (ultimate_scanner_v4.py). By moving beyond standard connect_ex() sockets to direct Scapy packet crafting, the tool achieved half-open SYN stealth scanning and active TTL-based OS fingerprinting. Thread pooling ensured stable execution, while the automated reporting module delivered structured output across multiple file formats, including an offline dark-mode HTML dashboard. This work demonstrates the practical implementation of low-level protocol

References

• [1] MITRE Corporation. (2026). Active Scanning: Technique T1595. MITRE ATT&CK Framework. Retrieved from https://attack.mitre.org/techniques/T1595/

•[2] Python Software Foundation. (2026). concurrent.futures — Launching parallel tasks. Python 3 Documentation. Retrieved from https://docs.python.org/3/library/concurrent.futures.html

• [3] Scapy Community. (2026). Scapy v2.5 Documentation: Packet Crafting & Layer 3/4 Scanning. Retrieved from https://scapy.readthedocs.io/

• [4] SEED Labs. (n.d.). SEED Ubuntu Virtual Machine Architecture. Syracuse University. Retrieved from https://seedsecuritylabs.org/

Appendices

(Note: This section contains the extended materials that would otherwise clutter the main body of the report.)

Appendix A: Full Source Codes (Ultimate_scanner__V2.py)

Appendix A

```
#!/usr/bin/env python3
"""
Ultimate Scanner v4
Features:
- Scapy TCP SYN Half-Open Stealth Scanning
- ThreadPoolExecutor for rate limiting
- Local/Offline Banner-to-CVE Mapping
- Scapy TCP/IP Stack OS Fingerprinting
- Automated Reporting: Markdown, CSV, TXT, and Interactive Dark-Mode HTML Dashboard
"""

import os
import sys
import csv
import socket
import datetime
import logging
from concurrent.futures import ThreadPoolExecutor

# Mute Scapy logging output
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
```

```
try:
    from scapy.all import IP, TCP, sr1, send, conf
    conf.verb = 0
    SCAPY_AVAILABLE = True
except ImportError:
    SCAPY_AVAILABLE = False

# Local Vulnerability Database (Offline Fallback)
LOCAL_VULN_DB = {
    "apache/2.4.49": "CVE-2021-41773 (Path Traversal / RCE)",
    "vsftpd 2.3.4": "CVE-2011-2523 (Backdoor Command Execution)",
    "openssh 7.2p2": "CVE-2016-6210 (User Enumeration)",
    "openssh 4.7p1": "CVE-2008-0166 (OpenSSL Weak Key Predictability)",
    "apache/2.2.8": "CVE-2008-0455 (HTTP Response Splitting)",
    "samba 3.0.20": "CVE-2007-2447 (Username Map Script Execution)"
}

class StealthOSFingerprinter:
    """Performs OS identification via TCP/IP packet header analysis."""

    @staticmethod
    def detect_os(target_ip, port=80):
        if not SCAPY_AVAILABLE:
            return {"os": "Scapy Unavailable", "ttl": "N/A", "window": "N/A"}

        try:
            probe = IP(dst=target_ip) / TCP(dport=port, flags="S")
            reply = sr1(probe, timeout=2)
```

```
        if reply is None:
            return {"os": "Filtered / No Response", "ttl": "N/A", "window": "N/A"}

        if reply.haslayer(TCP):
            ttl = reply[IP].ttl
            window = reply[TCP].window

            if ttl <= 64:
                os_family = "Linux / Unix / macOS"
            elif ttl <= 128:
                os_family = "Windows System"
            elif ttl <= 255:
                os_family = "Cisco / Network Device"
            else:
                os_family = "Unknown Stack"

            return {"os": os_family, "ttl": ttl, "window": window}
    except Exception as e:
        return {"os": f"Error: {str(e)}", "ttl": "N/A", "window": "N/A"}

    return {"os": "Unknown", "ttl": "N/A", "window": "N/A"}

class StealthScanner:
    """TCP SYN Stealth Scanner with Banner Grabbing & Offline CVE Matching."""

    def __init__(self, target_ip, ports, max_threads=10):
        self.target_ip = target_ip
```

```
    self.ports = ports
    self.max_threads = max_threads

def grab_banner(self, port):
    """Standard banner grabbing post-discovery."""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(1.5)
        s.connect((self.target_ip, port))
        s.sendall(b"HEAD / HTTP/1.1\r\nHost: target\r\n\r\n")
        banner = s.recv(1024).decode(errors="ignore").strip().split("\n")[0]
        s.close()
        return banner if banner else "No banner returned"
    except Exception:
        return "No banner captured"

def match_vulnerability(self, banner):
    """Cross-references banners with local CVE database."""
    banner_lower = banner.lower()
    matched_cves = []
    for service, cve in LOCAL_VULN_DB.items():
        if service in banner_lower:
            matched_cves.append(cve)
    return ", ".join(matched_cves) if matched_cves else "No known local CVE match"

def syn_scan_port(self, port):
    """Performs a TCP SYN Half-Open Stealth Scan on a specific port."""
    if not SCAPY_AVAILABLE:
```

```
        return self._fallback_connect_scan(port)

    try:
        syn_pkt = IP(dst=self.target_ip) / TCP(dport=port, flags="S")
        response = sr1(syn_pkt, timeout=1.5)

        if response is not None and response.haslayer(TCP):
            if response[TCP].flags == 0x12:  # SYN-ACK
                rst_pkt = IP(dst=self.target_ip) / TCP(dport=port, flags="R")
                send(rst_pkt)

                banner = self.grab_banner(port)
                vuln_info = self.match_vulnerability(banner)

                return {
                    "port": port,
                    "status": "OPEN (SYN-ACK)",
                    "banner": banner,
                    "vulnerabilities": vuln_info
                }
            elif response[TCP].flags == 0x14:  # RST
                return None
    except Exception:
        pass
    return None

def _fallback_connect_scan(self, port):
    try:
```

```
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(1.0)
            if s.connect_ex((self.target_ip, port)) == 0:
                banner = self.grab_banner(port)
                vuln_info = self.match_vulnerability(banner)
                s.close()
                return {
                    "port": port,
                    "status": "OPEN (Connect)",
                    "banner": banner,
                    "vulnerabilities": vuln_info
                }
            s.close()
        except Exception:
            pass
        return None

    def execute_scan(self):
        print(f"[*] Initiating SYN Stealth Scan on {self.target_ip} (Threads:
{self.max_threads})...")
        open_results = []

        with ThreadPoolExecutor(max_workers=self.max_threads) as executor:
            scanned_ports = executor.map(self.syn_scan_port, self.ports)

        for result in scanned_ports:
            if result:
                open_results.append(result)
```

```
            print(f"  [+] Port {result['port']}: {result['status']} | Banner: {result['banner']}")

        return open_results

class AutomatedReportGenerator:
    """Generates structured CSV, Markdown, TXT, and HTML reports."""

    def __init__(self, target_ip, results, os_data, output_dir="reports"):
        self.target_ip = target_ip
        self.results = results
        self.os_data = os_data
        self.output_dir = output_dir
        self.timestamp = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")

        if not os.path.exists(self.output_dir):
            os.makedirs(self.output_dir)

    def export_all(self):
        md_path = self._export_markdown()
        csv_path = self._export_csv()
        txt_path = self._export_txt()
        html_path = self._export_html()
        return md_path, csv_path, txt_path, html_path

    def _export_markdown(self):
        path = os.path.join(self.output_dir, f"report_{self.target_ip}_{self.timestamp}.md")
        with open(path, "w") as f:
            f.write(f"# Reconnaissance Report: {self.target_ip}\n\n")
```

```
            f.write(f"- **Scan Date:** {self.timestamp}\n")
            f.write(f"- **OS Prediction:** {self.os_data.get('os')} (TTL:
{self.os_data.get('ttl')})\n\n")
            f.write("## Discovered Services & Vulnerabilities\n\n")
            f.write("| Port | Status | Banner | Identified Vulnerabilities |\n")
            f.write("|------|--------|--------|----------------------------|\n")
            for r in self.results:
                f.write(f"| {r['port']} | {r['status']} | {r['banner']} | {r['vulnerabilities']} |\n")
        return path

    def _export_csv(self):
        path = os.path.join(self.output_dir, f"report_{self.target_ip}_{self.timestamp}.csv")
        with open(path, "w", newline="") as f:
            writer = csv.writer(f)
            writer.writerow(["Target", "Port", "Status", "Banner", "Vulnerabilities",
"Predicted_OS", "TTL", "Timestamp"])
            for r in self.results:
                writer.writerow([
                    self.target_ip, r['port'], r['status'], r['banner'],
                    r['vulnerabilities'], self.os_data.get('os'), self.os_data.get('ttl'), self.timestamp
                ])
        return path

    def _export_txt(self):
        path = os.path.join(self.output_dir, f"report_{self.target_ip}_{self.timestamp}.txt")
        with open(path, "w") as f:
            f.write("=" * 60 + "\n")
            f.write(f" RECONNAISSANCE SCAN REPORT - {self.target_ip}\n")
            f.write(f" Generated: {self.timestamp}\n")
```

```
            f.write("=" * 60 + "\n\n")
            f.write(f"Target OS Family: {self.os_data.get('os')}\n")
            f.write(f"Captured TTL:    {self.os_data.get('ttl')}\n\n")
            f.write("Open Port Findings:\n")
            f.write("-" * 60 + "\n")
            for r in self.results:
                f.write(f"Port {r['port']} [{r['status']}]\n")
                f.write(f"  Banner: {r['banner']}\n")
                f.write(f"  CVEs:   {r['vulnerabilities']}\n\n")
        return path

    def _export_html(self):
        path = os.path.join(self.output_dir, f"report_{self.target_ip}_{self.timestamp}.html")

        rows_html = ""
        for r in self.results:
            vuln_style = "color: #ff5555; font-weight: bold;" if "CVE" in r['vulnerabilities'] else
"color: #a6adc8;"
            rows_html += f"""
            <tr>
                <td><span class="badge port">{r['port']}</span></td>
                <td><span class="badge open">{r['status']}</span></td>
                <td><code>{r['banner']}</code></td>
                <td style="{vuln_style}">{r['vulnerabilities']}</td>
            </tr>
            """

        html_content = f"""<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Recon Report - {self.target_ip}</title>
  <style>
    body {{ background-color: #1e1e2e; color: #cdd6f4; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; margin: 0; padding: 40px; }}
    .container {{ max-width: 1000px; margin: 0 auto; background: #181825; padding: 30px; border-radius: 12px; box-shadow: 0 8px 24px rgba(0,0,0,0.5); }}
    h1 {{ color: #89b4fa; border-bottom: 2px solid #313244; padding-bottom: 10px; margin-top: 0; }}
    .cards {{ display: flex; gap: 20px; margin-bottom: 30px; }}
    .card {{ flex: 1; background: #313244; padding: 15px; border-radius: 8px; border-left: 4px solid #89b4fa; }}
    .card h3 {{ margin: 0 0 5px 0; font-size: 14px; color: #a6adc8; text-transform: uppercase; }}
    .card p {{ margin: 0; font-size: 18px; font-weight: bold; color: #f5e0dc; }}
    table {{ width: 100%; border-collapse: collapse; margin-top: 10px; }}
    th, td {{ padding: 12px 15px; text-align: left; border-bottom: 1px solid #313244; }}
    th {{ background-color: #313244; color: #89b4fa; text-transform: uppercase; font-size: 12px; letter-spacing: 1px; }}
    tr:hover {{ background-color: #2a2b3c; }}
    code {{ background: #11111b; padding: 3px 6px; border-radius: 4px; font-family: monospace; color: #a6e3a1; }}
    .badge {{ padding: 4px 8px; border-radius: 4px; font-size: 12px; font-weight: bold; }}
    .badge.port {{ background: #45475a; color: #f5e0dc; }}
    .badge.open {{ background: #a6e3a1; color: #11111b; }}
  </style>
</head>
<body>
```

```
<div class="container">
  <h1>Reconnaissance Report</h1>
  <div class="cards">
    <div class="card">
      <h3>Target Host</h3>
      <p>{self.target_ip}</p>
    </div>
    <div class="card">
      <h3>Predicted OS</h3>
      <p>{self.os_data.get('os')} (TTL: {self.os_data.get('ttl')})</p>
    </div>
    <div class="card">
      <h3>Open Ports</h3>
      <p>{len(self.results)} Detected</p>
    </div>
    <div class="card">
      <h3>Scan Timestamp</h3>
      <p>{self.timestamp}</p>
    </div>
  </div>
  <h2>Open Ports & Vulnerability Analysis</h2>
  <table>
    <thead>
      <tr>
        <th>Port</th>
        <th>Status</th>
        <th>Service Banner</th>
        <th>Vulnerabilities Detected</th>
```

```
            </tr>
          </thead>
          <tbody>
            {rows_html if rows_html else '<tr><td colspan="4">No open ports
detected.</td></tr>'}
          </tbody>
        </table>
    </div>
</body>
</html>
"""
        with open(path, "w") as f:
            f.write(html_content)
        return path

if __name__ == "__main__":
    DEFAULT_TARGET = "192.168.56.106"

    if len(sys.argv) > 1:
        target = sys.argv[1]
    else:
        target = DEFAULT_TARGET
        print(f"[*] No IP provided. Defaulting to SEED VM: {target}")

    ports = [21, 22, 23, 25, 53, 80, 110, 139, 443, 445, 3306, 8080]

    # Step 1: Execute Stealth SYN Scan
    scanner = StealthScanner(target_ip=target, ports=ports, max_threads=10)
```

```
results = scanner.execute_scan()

# Step 2: Perform OS Fingerprinting
target_port = results[0]['port'] if results else 80
print(f"[*] Performing Scapy OS Fingerprinting against port {target_port}...")
os_info = StealthOSFingerprinter.detect_os(target, port=target_port)
print(f"[+] Identified OS: {os_info['os']} (TTL: {os_info['ttl']})")

# Step 3: Generate Reports
print("[*] Exporting scan reports...")
reporter = AutomatedReportGenerator(target, results, os_info)
md_out, csv_out, txt_out, html_out = reporter.export_all()

print("\n[✓] Scan Complete. Structured reports generated successfully:")
print(f"  - Markdown:  {md_out}")
print(f"  - CSV:      {csv_out}")
print(f"  - Text Log:  {txt_out}")
print(f"  - HTML Dash: {html_out}")
```